Lect No.(11-13)

CSS Colors & Backgrounds, Box Model & Units

CSS Colors

CSS allows developers to define and apply colors to text, borders, and backgrounds in various ways, making web pages more attractive and consistent.

Ways to Define Colors

Color Names

Use predefined color names like:

```
 → red, blue, green, yellow, black, white, purple, etc.
h1 {
  color: blue;
}
```

HEX Values

Represent color in hexadecimal notation using `#RRGGBB`.

Examples:

```
 → #FF0000 = Red
 → #00FF00 = Green
 → #0000FF = Blue
p {
  color: #008080; /* Teal */
}
```

RGB Values

Defines color by mixing red, green, and blue light.

Format: `rgb(red, green, blue)`

Each value ranges from 0 to 255.

```
div {
  color: rgb(255, 0, 255); /* Magenta */
}
```

RGBA Values
Adds an alpha channel for transparency.
Format: `rgba(red, green, blue, alpha)`
Alpha value ranges from 0 (transparent) to 1 (opaque).

```
div {
  background-color: rgba(0, 128, 0, 0.5);
}
```

HSL and HSLA Values
HSL stands for Hue, Saturation, and Lightness.
→ Hue: 0–360° (color wheel)
→ Saturation: 0%–100%
→ Lightness: 0%–100%

```
p {
  color: hsl(240, 100%, 50%);
}
```

HSLA adds an alpha value for transparency.

```
p {
  color: hsla(240, 100%, 50%, 0.7);
}
```

CSS Backgrounds

CSS backgrounds make web designs more visually engaging. You can use colors, images, gradients, or multiple backgrounds.

Background Color
Sets a solid background color for an element.

```
body {
  background-color: lightblue;
}
```

Background Image
Adds an image as a background.

```
body {
  background-image: url("background.jpg");
}
```

Background Repeat

Defines if and how a background image repeats.
Options:

- `repeat` (default)
- `no-repeat`
- `repeat-x`
- `repeat-y`

```
body {
  background-image: url("pattern.png");
  background-repeat: no-repeat;
}
```

Background Position

Positions the image within an element.
Values: `top`, `center`, `bottom`, `left`, `right`, or exact coordinates.

```
body {
  background-position: center top;
}
```

Background Size

Determines how the image is scaled.
Values: `auto`, `cover`, `contain`, or specific dimensions.

```
body {
  background-size: cover;}
```

Background Attachment

Specifies whether the background scrolls with content or stays fixed.

```
body {
  background-attachment: fixed;}
```

Multiple Backgrounds
You can apply more than one background image.

```
div {
  background-image: url("layer1.png"), url("layer2.png");
  background-position: center, top right;
}
```

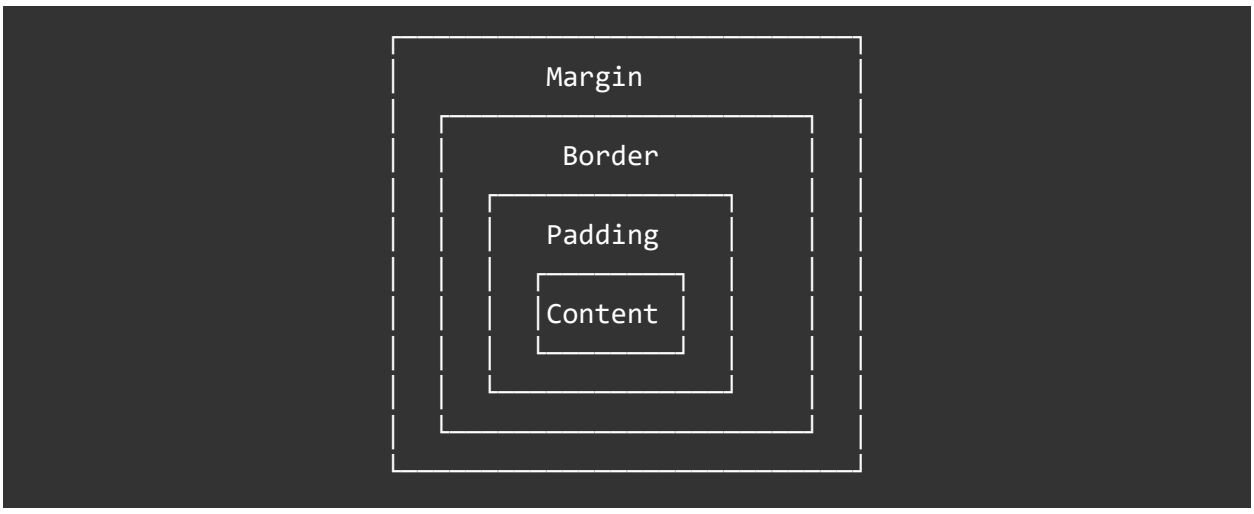
CSS Box Model

Every HTML element on a webpage is treated as a rectangular box. The **box model** explains how these boxes are structured and how spacing is calculated.

Structure of the Box Model

From the inside out:

Layer	Description	CSS Property
Content	The text or image inside the element	`width`, `height`
Padding	Space between content and border	`padding`
Border	Surrounds padding and content	`border`
Margin	Space outside the border	`margin`



Box Model Properties

Content

Defines the main area containing text or media.

```
div {
  width: 300px;
  height: 150px;
}
```

Padding

Adds space inside the element, between content and border.

```
div {
  padding: 20px;
  padding-top: 10px;
  padding-right: 15px;
}
```

Border

Surrounds the padding and content.

```
div {
  border: 2px solid black;
}
```

Margin

Adds space outside the element, separating it from others.

```
div {
  margin: 20px;
}
```

Box Sizing

Defines how total width and height are calculated.

Default (content-box):
Total size = width + padding + border

Border-box:
Width includes padding and border (better for layout control).

```
div {
  box-sizing: border-box;
}
```

Example: Box Model in Action

```
div {
  width: 200px;
  height: 100px;
  padding: 10px;
  border: 5px solid black;
  margin: 15px;
  background-color: lightcoral;
}
```

Total Width = 200 + 10 + 10 + 5 + 5 = 230px
Total Height = 100 + 10 + 10 + 5 + 5 = 130px

CSS Units

CSS units define the size of elements, text, and spacing. They can be **absolute** or **relative**.

Absolute Units

These have fixed values and do not change with screen size.

Unit	Meaning	Equivalent
`px`	Pixels	1 pixel
`cm`	Centimeters	1 cm

`mm`	Millimeters	1 mm
`in`	Inches	1 in = 2.54 cm
`pt`	Points	1 pt = 1/72 inch
`pc`	Picas	1 pc = 12 pt

```
p {
  font-size: 16px;
  margin: 1cm;
}
```

Relative Units

Relative units adapt to parent elements or viewport size, making designs responsive.

Unit	Description	Relative To
`em`	Relative to font-size of parent	Parent element
`rem`	Relative to root element (`html`)	Root font-size
`%`	Relative to parent container	Parent width/height
`vw`	1% of viewport width	Browser window width
`vh`	1% of viewport height	Browser window height
`vmin`	Smaller of `vw` and `vh`	Viewport
`vmax`	Larger of `vw` and `vh`	Viewport

```
div {
  font-size: 2em;
  padding: 5%;
  width: 50vw;
}
```

CSS Layout Basics and Positioning Elements

CSS Layout Basics

CSS Layout defines how elements are arranged and displayed on a web page.
It allows developers to control positioning, alignment, and spacing to create responsive and well-structured layouts.

The main goal of CSS layout is to control how text, images, and other content are placed within the browser window or container.

Key Display Modes in CSS

Block-Level Elements

- Always start on a new line.
- Take up the full width available.
- Examples: `<div>`, `<p>`, `<h1>`–`<h6>`, `<section>`, `<header>`.

```
div {
  display: block;
}
```

Inline Elements

- Do not start on a new line.
- Take up only as much width as necessary.
- Examples: `<span>`, `<a>`, `<strong>`, `<em>`.

```
span {
  display: inline;
}
```

Inline-Block Elements

- Behave like inline elements but allow setting width and height.

```
button {
  display: inline-block;
```

```
  width: 120px;
  height: 40px;
}
```

None (Hidden)

- The element will not be displayed on the page at all.

```
div {
  display: none;
}
```

Flow in Layout

Every element in a webpage follows the **normal document flow** unless changed by specific properties.

- **Normal Flow**: Elements appear in the order they are written (top to bottom, left to right).
- **Out of Flow**: Occurs when elements are floated, positioned, or flexed differently.

Common Layout Techniques

Float Layout
Float allows elements to be placed side by side, often used for older layouts before Flexbox and Grid.

```
.left {
  float: left;
  width: 50%;
}
.right {
  float: right;
  width: 50%;
}
```

To clear floated elements:

```css
.clearfix::after {
  content: "";
  display: block;
  clear: both;
}
```

Flexbox Layout

Flexbox provides a modern, flexible layout system for aligning and distributing space among items in a container.

```css
.container {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

Key Properties:

```css
display: flex;
flex-direction: row | column;
justify-content: center | space-between | space-around;
align-items: center | flex-start | flex-end;
gap: 10px;
```

Grid Layout

CSS Grid is a two-dimensional layout system used to design complete web layouts.

```css
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-gap: 20px;
}
```

Key Properties:

```css
display: grid;
grid-template-columns / grid-template-rows
gap or grid-gap
```

```
grid-column / grid-row
```

Responsive Layouts with Media Queries

Media queries allow layouts to adapt based on screen size.

```
@media (max-width: 768px) {
  .container {
    flex-direction: column;
  }
}
```

CSS Positioning Elements

CSS positioning is used to move elements precisely within a page.
It changes the **normal document flow** to control where and how elements appear.

Position Property

The `position` property defines the type of positioning used for an element.
Common values are:

- `static`
- `relative`
- `absolute`
- `fixed`
- `sticky`

Static Position

- Default positioning for all elements.
- Elements are placed according to the normal document flow.
- `top`, `bottom`, `left`, and `right` properties have no effect.

```
div {
  position: static;}
```

Relative Position

- Element is positioned relative to its **original position**.
- Does **not** remove the element from the normal flow.

```
div {
  position: relative;
  top: 20px;
  left: 30px;
}
```

The element moves 20px down and 30px right from where it would normally appear.

Absolute Position

- Element is positioned **relative to its nearest positioned ancestor** (not static).
- Removed from the normal document flow.

```
div {
  position: absolute;
  top: 50px;
  left: 100px;
}
```

If there's no positioned ancestor, the element positions itself relative to the `<html>` (page body).

Fixed Position

- Element is positioned relative to the **viewport** (browser window).
- It stays in the same place even when the page is scrolled.

```
nav {
  position: fixed;
  top: 0;
  width: 100%;
  background-color: #333;
}
```

Common use: sticky navigation bars or floating buttons.

Sticky Position

- A hybrid between **relative** and **fixed**.
- Acts like relative until a certain scroll position is reached, then “sticks” like fixed.

```
header {
  position: sticky;
  top: 0;
  background-color: white;
}
```

Sticky elements are often used for headers or menus that remain visible during scrolling.

Z-Index Property

Controls the **stacking order** of overlapping elements.
Elements with higher `z-index` appear above those with lower values.

```
.box1 {
  position: absolute;
  z-index: 1;
}

.box2 {
  position: absolute;
  z-index: 2; /* appears above box1 */
}
```

Top, Right, Bottom, Left

Used to position elements when `position` is not static.

```
div {
  position: absolute;
  top: 20px;
  right: 40px;
}
```

Example: Combining Positioning

```
<div class="container">
  <div class="box box1">Box 1</div>
  <div class="box box2">Box 2</div>
</div>
.container {
  position: relative;
  width: 400px;
  height: 200px;
  background: lightgray;
}
.box1 {
  position: absolute;
  top: 20px;
  left: 30px;
  background: coral;
  width: 100px;
  height: 100px;
}
.box2 {
  position: absolute;
  bottom: 20px;
  right: 30px;
  background: teal;
  width: 100px;
  height: 100px;
}
```

In this example:

- The container defines a reference point (relative).
- Box 1 and Box 2 are positioned absolutely inside it.

Float vs Position

Feature	Float	Position
Used for	Wrapping text around elements	Precise element placement
Flow	Affects document flow	Can remove element from flow
Control	Limited	High control (top, left, z-index)
Example	Text wrapping around images	Fixed header, pop-ups, overlays